

PARADIGMA ORIENTADO A OBJETOS

UNIDAD N° 2

Comportamiento de Objetos



SEMANA 3

Consideraciones previas

El contenido que se expone a continuación está ligado a los siguientes objetivos:

- Distingue estructuras de colección de datos como fuente de almacenamiento temporal de información dentro de una aplicación.
- Distingue el concepto de herencia de clases dentro de la aplicación (extends).
- Distingue el concepto de interface como clase contenedora de métodos y atributos comunes entre clases (implements).
- Distingue el concepto de conducta aleatoria en programación orientada a objeto.
- Distingue los métodos estáticos y dinámicos para establecer diferencias y aplicaciones.

Sobre las fuentes utilizadas en el material

El presente Material de Estudio constituye un ejercicio de recopilación de distintas fuentes, cuyas referencias bibliográficas estarán debidamente señaladas al final del documento. Este material, en ningún caso pretende asumir como propia la autoría de las ideas planteadas. La información que se incorpora tiene como única finalidad el apoyo para el desarrollo de los contenidos de la unidad correspondiente, respetando los derechos de autor ligados a las ideas e información seleccionada para los fines específicos de cada asignatura.

Introducción

Estamos en la tercera semana de la asignatura, y marca el inicio de una nueva unidad. Esta sección se llama “Comportamiento de objetos”, y busca la creación de aplicaciones usando colecciones que permitan almacenar datos, usando a la vez los conceptos de herencia y polimorfismo.

En esta ocasión corresponderá conocer las distintas maneras de almacenar datos en memoria primaria usando estructuras fijas y dinámicas. Al mismo tiempo se analizará a fondo el concepto de clase padre y clase hijo, y la diferencia entre los métodos estáticos y dinámicos. Por último, se analizará el uso de datos aleatorios en diversos programas.

Como ha sido la tónica en semanas anteriores, se utilizará como medio de aprendizaje el programa Greenfoot. Si bien su uso principal no se asocia al estudio de estructuras como arreglos y matrices, se puede igualmente utilizar en este aspecto ya que todos los elementos propios del lenguaje Java son soportados por este aplicativo.

Ideas fuerza

1. **Matriz:** es un contenedor de datos en memoria principal que se caracteriza porque todos sus elementos son del mismo tipo. Pueden ser unidimensionales o multidimensionales.
2. **Arreglo:** para efectos de la programación en Java, un arreglo o array se puede considerar un sinónimo de matriz.
3. **Lista:** es una estructura que permite almacenar datos u objetos del mismo tipo, con la característica que su comportamiento es dinámico.
4. **Herencia:** es uno de los cuatro pilares de la programación orientada a objetos, y consiste en la capacidad que tiene una clase de “heredar” desde una clase padre todos sus atributos y métodos.
5. **Interface:** Una interface o interfaz en Java es una colección de métodos abstractos y propiedades constantes. En ellas se especifica qué se debe hacer, pero no su implementación.
6. **Conducta aleatoria:** es una acción que se desarrolla en base a un evento al azar, vale decir, que puede darse con una cierta probabilidad.

Índice

Contenido

<i>1.- Almacenamiento de datos.....</i>	<i>6</i>
1.1.- Arreglos.....	6
1.2.- Arreglos multidimensionales	8
1.3.- Listas	8
<i>2.- Herencia.....</i>	<i>10</i>
2.1.- Extends.....	11
2.2.- Interfaces	12
2.3.- Sentencia super	12
<i>3.- Métodos estáticos y dinámicos</i>	<i>13</i>
<i>4.- Conductas aleatorias</i>	<i>14</i>
4.1.- Método Math.random().....	14
4.2.- Clase java.util.random	15

Desarrollo

1.- Almacenamiento de datos

En ocasiones es conveniente poder almacenar y referenciar variables que representan una colección de valores, de forma que sea posible tratarlos de manera uniforme.

En el lenguaje Java (así como en la mayoría de los lenguajes de programación) se introduce el concepto de arreglo o colección de valores del mismo tipo, con nombre común, y referenciables uno a uno, a través de un índice.

Dado que se pueden almacenar datos en este tipo de estructuras tanto de manera unidimensional como multidimensional, son una estructura de datos muy útil para muchos desarrollos. Sin embargo, su inconveniente es que la cantidad de datos a almacenar es fija, y se define al momento de declararlas. Frente a esto aparecen las listas, que son estructuras de tipo dinámica. En esta sección se analizará ambos tipos.

1.1.- Arreglos

Un arreglo o array es una colección de componentes homogénea, esto es, todas sus componentes son del mismo tipo de datos, de un tamaño predefinido, con un nombre o identificador.

Las características principales de una variable array son:

- Es posible acceder y modificar cada una de las componentes individuales por su posición dentro del grupo o colección en tiempo constante.
- El número de componentes de un array se establece inicialmente, no siendo posible su modificación posterior, salvo volviendo a construir el array de nuevo.

Mediante la declaración de un array se define, en primer lugar, el tipo de cada una de sus componentes de la forma:

```
tipoBase[] elArray;
```

En el caso anterior, tipoBase es cualquier tipo de datos, ya sea elemental o referencia. Por ejemplo:

```
double[] tempMed;  
String[] texto;  
Punto[] camino;
```

Para establecer el número de elementos del array es necesaria una operación explícita que establezca inicialmente dicho número, lo que se consigue haciendo uso del operador de construcción, new, que permite asignar espacio. El operador new recibe el número (num, un entero no negativo) y tipo (tipoBase) de los elementos del array, de la siguiente manera:

```
//Esta sería la estructura base.  
elArray = new tipoBase [num];  
  
//Estos son ejemplos de arreglos de diferentes tipos  
double[] tempMed;  
tempMed = new double [31];  
String[] texto;  
texto = new String[100];  
Punto[] Camino;  
camino = new Punto [20];
```

Cada una de las componentes del array se indexan con valores entre 0 y elArray.length - 1, ambos incluidos, con la siguiente sintaxis y representado gráficamente como se muestra a continuación:

```
// 0 <= expInt < elArray.length  
tipoBase componente = elArray[expInt];
```

En el caso anterior, expInt es una expresión numérica entera que se evalúa entre 0 y la longitud del array menos uno, De no ser así, ocurrirá un error durante la compilación ya que se busca acceder a un elemento que no existe.

1.2.- Arreglos multidimensionales

Las componentes de un array pueden ser, a su vez, otros arrays, Esto genera estructuras que permiten almacenar datos en dos o más coordenadas. Si bien no hay un límite para la cantidad de dimensiones, por lo general se trabaja con una o dos, más que ello puede ser complejo.

La sintaxis de Java para declarar un array multidimensional sería:

```
tipoBase [...] elArrayMulti;
```

Por ejemplo, el código siguiente declara m1, m2 y m3 como arrays bidimensionales; sin embargo, aunque las tres sean correctas, se utilizará la primera forma de definición que es la recomendada por los creadores del lenguaje.

```
double[][] m1; // Válido  
double[] m2[]; // Válido, no recomendado.  
double m3[][]; // Válido, no recomendado.
```

Para crear un array bidimensional se utiliza el operador new (igual que en el caso unidimensional). Por ejemplo, la instrucción siguiente declara y crea en Java un array bidimensional o matriz de 4X4 elementos de tipo double, representado en memoria como se muestra a continuación:

```
double[][] matriz = new double[4][4];
```

Para acceder a una posición de la matriz se debe indicar dos coordenadas. En un arreglo bidimensional, el primer índice se conocerá como la fila y el segundo como la columna, aunque esto es solo una convención. Si el array tiene n filas y m columnas, el valor de las filas fluctuará entre 0 y n-1, mientras que el de las columnas lo hará entre 0 y m-1. En el ejemplo anterior, tanto las filas como las columnas se moverán entre 0 y 3, ya que sería una matriz cuadrada (la cantidad de filas es la misma que la de las columnas).

1.3.- Listas

Las listas [2] son estructuras que se declaran de manera similar a una variable o arreglo, que permiten almacenar datos. La gran diferencia con los arreglos es que

las listas no tienen un límite, y es posible agregar, eliminar y modificar datos sin un límite de acciones.

Si bien Java contiene varios tipos de listas, en este caso el estudio se centrará en un tipo de ellos, la clase ArrayList. Esta clase se usa para trabajar colecciones de objetos. La clase ArrayList pertenece al paquete java.util. Permite representar un arreglo de objetos de cualquier tipo, incluyendo variables elementales y datos estructurados. Al mismo tiempo permite acceder a los elementos por medio de un índice.

Para crear e inicializar un ArrayList se debe usar un comando como el que se indica:

```
ArrayList<tipoDato> miArray = new ArrayList<tipoDato>();
```

En el ejemplo anterior, “tipoDato” es el tipo de element que se guardará en el ArrayList. Es importante recordar que no es necesario en este caso indicar una cantidad de elementos, ya que son elementos de largo variable, sin un límite predeterminado. Además, es posible agregar nuevos elementos, modificar los existentes y eliminarlos. Dado que ArrayList es una clase, tiene métodos que se pueden utilizar y que facilitan la labor del programador; en la tabla siguiente se indican los más usados.

Retorno	Método	Descripción
boolean	add(E elemento)	Agrega el objeto o elemento del parámetro al final del ArrayList.
void	add(int índice, E elemento)	Inserta el objeto del segundo parámetro en la posición dada por el primer parámetro.
E (elemento)	get(int indice)	Retorna el objeto de la posición especificada en el parámetro.
int	indexOf(Object o)	Retorna el índice de la primera ocurrencia del objeto del parámetro o retorna -1 si el ArrayList no contiene dicho objeto.
boolean	isEmpty()	Retorna verdadero si el ArrayList no contiene elementos.
boolean	remove(Object o)	Remueve la primera ocurrencia del objeto especificado desde el ArrayList. Si el ArrayList no contiene el elemento retorna falso.

E (elemento)	remove(int indice)	Elimina y retorna el objeto de la posición especificada en el parámetro.
int	size()	Retorna el número de objetos que existe en el ArrayList.

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

Se necesita crear un programa en donde se registren todas las temperaturas promedio en grados Celsius de todos los días de un mes cualquiera. ¿Qué estructura de las que se han visto sería mejor usar?

Se desea además tener registro de la cantidad de milímetros de agua caída en cada día del año en curso. ¿Qué estructura sería más conveniente usar en términos de espacio utilizado y de acceso a los datos?



2.- Herencia

La herencia es el concepto que permite que se puedan definir nuevas clases basadas en clases existentes, con el fin de reutilizar el código previamente desarrollado, generando una jerarquía de clases dentro de la aplicación.

Como consecuencia de lo anterior, si una clase se deriva de otra, se dice que esta hereda sus atributos y métodos. La clase derivada puede añadir nuevos atributos y

métodos y/o redefinir los atributos y métodos heredados. Para que un atributo y método puedan ser heredados es necesario que su visibilidad sea “protected”.

En Java, a diferencia de otros lenguajes orientados a objetos, una clase solo puede derivar de una única clase, con lo cual, no es posible realizar herencia múltiple con base en clases.

2.1.- Extends

La sentencia “extends” permite implementar el concepto de herencia. Se incluye para que una clase herede de otra clase.

Por ejemplo, en un contexto educativo todos quienes pertenecen a la comunidad académica, que podría estar representada por una clase Persona. Un Estudiante, al ser parte de la comunidad académica, debe heredar o “extender” los atributos y métodos que tenga la clase Persona, y por cierto tener sus propios atributos y métodos que lo identifiquen de las demás clases.

La sintaxis básica del caso anterior es la que se indica a continuación.

```
public class Persona{  
    ...  
}  
  
public class Estudiante extends Persona{  
    ...  
}
```

Para estos efectos es necesario definir los siguientes conceptos:

- Clase padre: es la clase que origina la herencia, vale decir, clase que es heredada o extendida por una o más clases. En el ejemplo anterior la clase padre sería Persona.
- Clase hija: corresponde a la clase que ha heredado o extendido otra clase; en el ejemplo anterior la clase Estudiante cumpliría este rol. Es importante además tener las siguientes consideraciones:
 - Una clase hija puede ser a la vez clase padre de otra

- Una clase hija solo puede tener una clase padre, vale decir, en el lenguaje Java no se permite la herencia múltiple
- La herencia se da a múltiples niveles. Bajo esta premisa, en el ejemplo anterior a partir de la clase Estudiante se pueden generar otras clases, las que heredarán los atributos y métodos de su clase padre y a la vez de la clase Persona.

2.2.- Interfaces

En programación orientada a objetos, una interfaz [2] o interface es un tipo especial de clase que permite realizar un conjunto de declaraciones de métodos sin implementación. En una interfaz también se pueden definir constantes que son implícitamente public, static y finaly; estas deben siempre inicializarse en la declaración.

Para que una clase use las definiciones de una interfaz, dicha clase debe incluir la sentencia “implements” la cual indica que implementa la interfaz.

```
public interface MiInterfaz {  
    ...  
}  
  
public class Miclase implements MiInterfaz {  
    ...  
}
```

El objetivo de los métodos declarados en una interfaz es definir un tipo de conducta para las clases que implementan dicha interfaz. Todas las clases que colocan en funcionamiento una determinada interfaz están obligadas a proporcionar una implementación de los métodos declarados en la interfaz adquiriendo un comportamiento.

Una clase puede implementar una o varias interfaces. Para indicar que una clase implementa más de una interfaz, se ponen los nombres de las interfaces separadas por comas, posterior a incluir la sentencia “implements”.

2.3.- Sentencia super

La sentencia "super" es utilizada para acceder a métodos implementados en la clase superior en el concepto de herencia. Esta sentencia es comúnmente utilizada para acceder al constructor de la clase superior desde el constructor de la clase inferior.

Por ejemplo, en el ejemplo antes visto existe una clase Persona con atributos posibles como identificación, nombre, apellido y correo, y una clase Estudiante que puede acceder a estos atributos pero que adicionalmente, tiene atributos como código y facultad. Al implementar el constructor de la clase estudiante para asignar los valores de los atributos, se puede hacer un llamado al constructor de la clase persona enviándole los parámetros definidos en dicha clase.

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

*¿Cuáles son las mayores diferencias que existen entre el comando
implements del comando extends?*



3.- Métodos estáticos y dinámicos

En semanas anteriores se había mencionado que un método es un segmento de código, debidamente encapsulado y parametrizado, que se puede usar en otras partes de un programa, produciendo un valor resultante o teniendo algún efecto sobre los datos o en la ejecución del programa.

Dentro de todos los tipos de métodos que existen, Java distingue entre métodos de objeto o dinámicos y métodos de clase o estáticos:

- Los métodos de objeto son aquellos que se definen en una clase para crear y manipular la información de un objeto de la clase. Se distingue entre:
 - Métodos constructores: Permiten crear los objetos de una clase usando el operador “new”. Son fundamentales en las clases y existen por defecto.
 - Métodos de instancia: Permiten manipular la información de un objeto de la clase previamente creado, aplicándoselos mediante la notación especial de “.”.
- Los métodos de clase, por el contrario, son los métodos que no se aplican sobre un objeto de la clase. Se deben declarar como “static”, de ahí que se les llame también métodos estáticos (y, en contraposición, a los métodos de objeto se les llame métodos dinámicos). Las clases suelen venir acompañadas por una documentación que da información acerca de los métodos que proporciona cada una de ellas.

4.- Conductas aleatorias

Una conducta aleatoria es un hecho que ocurre o se genera con un cierto grado de incertidumbre. No se puede anticipar el resultado de este tipo de situaciones. En el caso del lenguaje Java, existen dos maneras de generar números aleatorios.

4.1.- Método Math.random()

La clase Math pertenece al lenguaje de programación Java, y se utiliza para utilizar funciones que realizan cálculos matemáticos complejos, como funciones trigonométricas y estadísticas. Esta clase evita que los desarrolladores deban crear funciones complejas para resolver problemas que son comunes a muchos proyectos.

Una opción es utilizar la clase Math; si se opta por la usar la función random() de esta clase el proceso es muy sencillo ya que esta función devuelve un número aleatorio entre 0 y 1 (de tipo double) y solo es necesario multiplicarlo por el número

que será el límite superior y/o sumarle (o restarle) algún número para que el límite inferior sea otro distinto de 0.

Esta función es estática, por lo que no es necesario instanciar un objeto de la clase Math para usarlo. Al mismo tiempo, no es necesario

Un ejemplo de uso de este método es el siguiente:

```
int numero = (int)(Math.random()*10+1);
```

En el ejemplo anterior el valor generado estará entre 1 y 10; esto sucede porque primero se multiplica el resultado del método random() (un valor decimal entre 0 y 1) por 10, con lo que se tendría un número entre 0 y 9. Al sumarle 1 al resultado obtenido se tendrá finalmente un número entre 1 y 10.

Ahora bien, ¿cómo se debería actuar si se desea un número aleatorio entre dos valores? Por ejemplo, si se desea obtener números entre 25 y 75, el resultado del random() se debe multiplicar por 51, obteniendo números entre 0 y 51) y se le sumaría 25 para obtener el resultado esperado. La fórmula para obtener un número entre dos números X e Y cualquiera es $(X-Y+1)+Y$, donde X es el número mayor del rango e Y es el número menor.

```
int numero = (int)(Math.random()*(X-Y+1)+Y;  
int numero = (int)(Math.random()*(75-25+1)+25);
```

4.2.- Clase java.util.random

Una segunda opción es hacer uso de la clase java.util.random. Al igual que la anterior, también pertenece a las clases nativa del lenguaje de programación; sin embargo, para poder usarla se debe importar en la sección superior de la clase, antes de la declaración de la clase.

El comando para generar un número aleatorio entero entre 0 (inclusive) y n-1 es el siguiente:

```
// Crea un objeto de la clase Random  
Random randomno = new Random();
```

```
// Genera un número aleatorio  
int numero = randomno.nextInt(n);
```

La dinámica para generar números aleatorios es igual a la del método anterior, pero ofrece un mayor abanico de posibilidades en lo referente al tipo de los números generados (boolean, integer, double, float, Long y arrays de bytes), en la forma en la que se generan los números (distribución gaussiana) y también se permite pasarle una semilla para generar los números a partir de esta aunque hay que tener en cuenta que si es un número fijo se generarán los números siguiendo siempre la misma secuencia por lo que si se crean 2 instancias con la misma semilla ambas producirán la misma secuencia. Si no se indica ninguna semilla por defecto se usa `System.currentTimeMillis()`.

```
Random numAleatorio = new Random();  
  
// Constructor pasándole una semilla  
Random numAleatorio = new Random(234);  
  
// Número entero entre 25 y 75  
int n = numAleatorio.nextInt(75-25+1) + 25;  
  
// Asigna una nueva semilla  
numAleatorio.setSeed(1235);  
  
// Numero decimal entre 0.0 y 1.0  
double d = numAleatorio.nextDouble();
```

Es importante considerar en el ejemplo anterior que asignar una semilla al crear el objeto de la clase `Random` es de tipo opcional.

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

Usando los métodos y clases antes vistos ¿qué posibilidades existen de generar un carácter aleatorio?



Conclusión

En la tercera semana de la asignatura se ha iniciado la segunda unidad de esta, que lleva por nombre Comportamiento de Objetos. El objetivo de esta unidad es construir aplicaciones utilizando colecciones para el almacenamiento de datos e implementando herencia y polimorfismo según requerimientos y estándares de la industria.

Con la finalidad de cumplir esto, en este documento se ha revisado el concepto de colección y los tipos que existen en el lenguaje de programación Java. También se analizó el comando “extends” y su función en la herencia. Otro término analizado fue “implements”, que permite hacer uso de las interfaces.

También se comentó respecto de la diferencia entre los métodos estáticos y dinámicos, y su entendimiento desde una perspectiva práctica. Por último, se analizaron dos maneras diferentes de obtener valores aleatorios.

En la siguiente semana se continuará haciendo referencia a los tipos de acciones que dan vida a las clases, profundizando el concepto de polimorfismo y de listas, y el acceso a datos desde el teclado.

Referencias

[1] Prieto Saez, N. y Casanova Faus, A. (2016). Empezar a programar usando Java (3a. ed.). Valencia, Spain: Editorial de la Universidad Politécnica de Valencia.

[2] Flórez Fernández, H. A. (2012). Programación orientada a objetos usando java. Bogotá, Colombia: Ecoe Ediciones.

[3] Programando o Intentándolo, “Cómo generar números aleatorios en Java”
Referencia: <https://programandoointentandolo.com/2013/10/como-generar-numeros-aleatorios-en-java.html>

